

Guía de Referencia Completa: XML Schema (XSD)

Bloque 1: Estructura Raíz y Declaraciones Básicas

1. **<xs:schema>** (Elemento Raíz)

- **Definición:** Es el elemento raíz obligatorio de cualquier archivo XSD. Define que el documento es un esquema, establece el espacio de nombres estándar del W3C (**xmlns:xs**) y puede incluir configuraciones de destino.
- **Ejemplo:**

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">  
</xs:schema>
```

2. **<xs:element>** (Declaración de Elementos)

- **Definición:** Declara una etiqueta o elemento que puede aparecer en el documento XML asociado. Puede indicar directamente un tipo básico o asociarse a un tipo estructurado complejo.
- **Ejemplo:**

```
<xs:element name="to" type="xs:string" />  
<xs:element name="note" type="notaType" />
```

3. **<xs:attribute>** (Declaración de Atributos)

- **Definición:** Define un atributo que pertenecerá a un elemento específico. Los atributos siempre son de tipo simple y se configuran dentro de los tipos complejos.
- **Atributos clave:** **use="required"** (obligatorio) u **use="optional"** (opcional, por defecto). También admite **default** (valor por defecto) o **fixed** (valor fijo).
- **Ejemplo:**

```
<xs:attribute name="id" type="xs:string" use="required" />  
<xs:attribute name="pais" type="xs:string" fixed="España" />
```

4. `<xs:ref>` (Referencia a Elementos)

- **Definición:** Permite reutilizar un elemento que ya ha sido declarado de forma global (como hijo directo de `<xs:schema>`). Ayuda a mantener el código limpio y modular.
- **Ejemplo:**

```
<xs:element name="profesor" type="xs:string"/>
```

```
<xs:complexType name="institutoType">  
  <xs:sequence>  
    <xs:element ref="profesor" minOccurs="1" maxOccurs="unbounded"/>  
  </xs:sequence>  
</xs:complexType>
```

5. Tipos Base Estándar de XSD

Son los tipos de datos predefinidos más utilizados para validar el contenido de texto de elementos o atributos:

- `xs:string` - Cadena de texto genérica.
- `xs:float` / `xs:decimal` - Números de coma flotante y valores decimales precisos (ideal para precios).
- `xs:integer` / `xs:int` / `xs:positiveInteger` - Variantes para números enteros.
- `xs:boolean` - Valores lógicos (`true` o `false`, 1 o 0).
- `xs:gYear` - Representa un año específico en formato `YYYY`.
- `xs:date` - Representa una fecha en formato compuesto `YYYY-MM-DD`.
- **Ejemplo de uso:**

```
<xs:element name="año" type="xs:gYear" />  
<xs:element name="precio" type="xs:decimal" />
```

Bloque 2: Estructuras y Compositores de Contenido

6. `<xs:complexType>` (Tipos Complejos)

- **Definición:** Define un tipo de elemento estructurado. Se utiliza cuando un elemento va a contener elementos hijos, atributos, o una mezcla de ambos (`mixed="true"`).
- **Ejemplo:**

```
<xs:complexType name="personaType">
  <xs:sequence>
    <xs:element name="nombre" type="xs:string"/>
    <xs:element name="apellido" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

7. `<xs:sequence>` y otros Compositores de Contenido

Indican las reglas de orden y aparición de los elementos hijos dentro de un tipo complejo:

- `<xs:sequence>`: Los elementos hijos **deben** aparecer estrictamente en el orden exacto en el que están declarados.
- `<xs:choice>` (*Complemento clave de examen*): Funciona como una alternativa exclusiva; en el XML **solo uno** de los elementos declarados dentro del bloque puede aparecer.
- `<xs:all>` (*Complemento clave de examen*): Los elementos declarados pueden aparecer en **cualquier orden**, pero todos deben aparecer exactamente una vez (o ninguna, si se configuran opcionales).
- **Ejemplo:**

```
<xs:sequence>
  <xs:element name="to" type="xs:string" />
  <xs:element name="from" type="xs:string" />
</xs:sequence>
```

```
<xs:choice>
  <xs:element name="nif" type="xs:string" />
  <xs:element name="nie" type="xs:string" />
</xs:choice>
```

8. Atributos de Control de Ocurrencia: **minOccurs** y **maxOccurs**

- **Definición:** Controlan la cardinalidad. Indican cuántas veces como mínimo (**minOccurs**) y como máximo (**maxOccurs**) puede aparecer un elemento en esa posición del XML. Su valor por defecto es **1**.
- **Valor especial:** **unbounded** (indica que no hay límite superior de repeticiones).
- **Ejemplo:**

```
<xs:element name="marca" type="xs:string" minOccurs="0" />
```

```
<xs:element name="pagina" maxOccurs="unbounded" />
```

Bloque 3: Restricciones y Validaciones (Tipos Simples)

9. **<xs:simpleType>** (Tipos Simples Personalizados)

- **Definición:** Se utiliza para crear un tipo de dato propio a partir de un tipo base predefinido, aplicando restricciones (facetas) a su contenido. Un tipo simple **nunca** puede contener elementos hijos ni atributos.
- **Ejemplo:**

```
<xs:simpleType name="codigoType">  
  <xs:restriction base="xs:string">  
    <xs:minLength value="2" />  
    <xs:maxLength value="5" />  
  </xs:restriction>  
</xs:simpleType>
```

10. **<xs:restriction>** (Definición de Restricciones)

- **Definición:** Es el contenedor de las facetas de validación. Define cuál es el tipo de datos base (**base="..."**) que se va a limitar o acotar.
- **Ejemplo:**

```
<xs:restriction base="xs:integer">  
  </xs:restriction>
```

11. Facetas de Longitud: `<xs:minLength>` y `<xs:maxLength>`

- **Definición:** Restringen el número mínimo y máximo de caracteres permitidos en una cadena de texto (`xs:string`). Si se requiere una longitud fija exacta, se utiliza `<xs:length>`.
- **Ejemplo:**

```
<xs:minLength value="2" />
```

```
<xs:maxLength value="5" />
```

12. Faceta de Expresiones Regulares: `<xs:pattern>`

- **Definición:** Aplica una expresión regular estricta que el valor del texto debe cumplir exactamente para ser considerado válido. Muy útil para códigos de producto, matrículas, DNI o códigos postales.
- **Ejemplo:**

```
<xs:pattern value="P[0-9]{3}" />
```

Nota complementaria de tus apuntes: Para rangos numéricos, recuerda usar facetas similares como `<xs:minInclusive value="0"/>` y `<xs:maxInclusive value="100"/>` para acotar valores enteros o decimales.

Bloque 4: Modelos de Contenido Avanzado y Extensión

13. **<xs:simpleContent>** (Contenido Simple en Tipos Complejos)

- **Definición:** Se aplica exclusivamente dentro de un **<xs:complexType>**. Permite definir un elemento que **solo contiene texto plano** (u otro tipo simple base) pero que **añade atributos**.
- **Ejemplo:**

```
<xs:complexType name="precioType">
  <xs:simpleContent>
    <xs:extension base="xs:decimal">
      <xs:attribute name="moneda" type="xs:string" use="required" />
    </xs:extension>
  </xs:simpleContent>
</xs:complexType>
```

14. **<xs:extension>** (Mecanismo de Herencia)

- **Definición:** Extiende las capacidades de un tipo base existente. Si se usa dentro de **simpleContent**, añade atributos a un tipo de datos simple. Si se usa dentro de **complexContent** (su contraparte para estructuras complejas), permite simular herencia orientada a objetos agregando nuevos elementos hijos y atributos a un tipo complejo base.
- **Ejemplo:**

```
<xs:complexType name="personaAmpliadaType">
  <xs:complexContent>
    <xs:extension base="personaType">
      <xs:sequence>
        <xs:element name="ciudad" type="xs:string" />
      </xs:sequence>
      <xs:attribute name="codigoPostal" type="xs:string" />
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```